

Computer Graphics — Week 11

Drawing Implicit Functions

Prof: Eric Biagioli

Exercises to upload to Canvas

The following should be implemented in C or C++. You should have a **marching_squares.cpp** (or .c) and a **marching_cubes.cpp** file in the same directory. Before uploading to Canvas, you should ZIP this directory (not tar, use ZIP) and upload the new zip file with the name **{marching-your-full-name}.zip**

Part 1: Marching Squares

Implement a program that is able to draw an implicit curve, using **marching squares**.

The prototype of the function that you write should be the following:

Listing 1: Function prototype

```
void marching_squares(void (*fn)(float, float),
                     uint32_t xmin, uint32_t ymin, uint32_t xmax, uint32_t ymax,
                     uint32_t resolution,
                     float isovalue)
```

where:

- fn is the curve that we want to draw. You receive a pointer to this function
- $x_{\min}, y_{\min}, x_{\max}, y_{\max}$ defines the bottom-left and top-right corners of the bounding rectangle in which we will draw. You should setup your SVG to have a viewBox corresponding to this.
- **resolution** the number of horizontal and vertical points your grid should have. E.g: a resolution of 5 should create a 5x5 grid with 25 sample points.
- **isovalue** The "level" of the function we are looking for. This is the value we look for when comparing evaluated points on a line. You can also understand it as the value that defines the point in which something is "outside" or "inside".

Result

You should write an SVG file named **output.svg** to the same directory of your c++ program.

Part 2: Marching Cubes

Exactly the same as the previous problem, but this time in 3D. Instead of generating an SVG file, this time generate a PLY file. Notice that the number of cases for marching cubes will be 256 (not 16 anymore).

The prototype of the function that you write should be the following:

Listing 2: Function prototype

```
void marching_cubes(void (*fn)(float, float, float),
                   uint32_t xmin, uint32_t ymin, uint32_t zmin,
                   uint32_t xmax, uint32_t ymax, uint32_t zmax,
                   uint32_t resolution,
                   float isovalue)
```

where:

- fn is the curve that we want to draw. You receive a pointer to this function
- $x_{\min}, y_{\min}, z_{\min}, x_{\max}, y_{\max}, z_{\max}$ defines the bottom-left and top-right corners (opposite corners of a cube) of the bounding box in which we will draw.
- **resolution** the number of points your volume should have in each axis (x, y, z). E.g: a resolution of 5 should create a 5x5x5 volume with 125 sample points.
- **isovalue** The "level" of the function we are looking for. This is the value we look for when comparing evaluated points on a line. You can also understand it as the value that defines the point in which something is "outside" or "inside".

Result

You should write a PLY file named `output.ply` to the same directory of your c++ program.

A very good reference to implement Marching Cubes is found [Here](#)

What will be most useful are the two tables presented there, which encode the 256 cases.

Notes for Both Parts

Notice that both the marching squares and the marching cubes that are being asked are the simplest approach. The statements of the problems are not asking to make sure that your representations have no repeated points or similar.

The same happens with marching cubes. What is expected is, basically, a soup of polygons that happens to give the impression of forming a surface in 3D. It is perfectly possible to repeat several times the same points in the PLY.

More Exercises (not to be uploaded to Canvas, but will be part of the PC3)

It is easy to define boolean operators to combine implicit functions. Given that with implicit functions we can know if a region is inside/outside a curve/surface, we can easily

define boolean operations (intersection, union, subtraction, etc.) of regions defined by implicit functions.

The present problem consists of defining an evaluation function that will enable these boolean operations, and will allow us to easily draw much more complex shapes.

Reference (scroll to the Primitive Combinations section)